Learning Agents - Day 1

Day 1 - Make an LLM do something from your code

Section 1 - LLM → Token → Input → Output → Model → API

Text generation models are also known as generative pretrained transformers (GPTs) that have been trained to understand natural and formal language.
LLM — a model that takes a sequence of tokens and predicts/generates what comes next
Token — the chunks of text the model processes rather than literally processing words
Prompt/input — information/instructions, which form the "chunks of text" input that you send the model. Designing a prompt is how you program a model.

Embeddings
Embeddings are vector representations of a piece of data (eg. some text). The representation helps in preserving the context and the meaning of the data. So, chunks of data that are similar to each other will tend to have embeddings that are closer together than unrelated data.
OpenAI provides text embedding models such that it takes a text string as input and outputs an embedding vector. Embeddings are useful for search clustering, recommendations, anomaly detection, classification etc.

Tokens are - basically the chunks of data being used by Text generation models or Embedding models. Tokens represent a commonly occurring sequence of characters. For eg - the word "tokenization" is decomposed as "token" and "ization", whereas the word "the" is a single token. As a general rule of thumb, 1 token is approx 4 characters of English text.

A key limitation to keep in mind is that - for a text generation model - the prompt and output combined can not be more than the model's maximum context length

API
An API is a defined way for one piece of software to communicate with another piece of software. It is an agreed upon interface/rule that lets a software A request something from Software B and receives a response.

Main Source - https://developers.openai.com/api/docs/concepts

Section 2 : Developer Quickstart

API Key - API key helps in identifying/authenticating the application or program to the API developer (like OpenAI). It tells the developer that - "this program is authorized to access this developer/user account"

1. Create the API Key
2. Add it in the ~/.zshrc through the terminal
    a. To double check, run this in the terminal after :
    source ~/.zshrc

if [[-n "$OPENAI_API_KEY"]]; then echo "API key loaded"; else echo "API key missing"; fi

3. Install **Python** = the programming language you're writing your application in.
4. Install **OpenAI Python SDK** = a library someone wrote **in/for Python** that makes communicating with OpenAI's API easy.
    a. python3 -m pip install openai
5. Without the SDK, Python can still call the OpenAI API, but you'd have to manually construct HTTP requests, headers, authentication, parse responses, etc.

Section 3 : Build

Now you're ready to code your first program that talks to the OpenAI model via the OpenAI API.

Here's a sample program :

```
from openai import OpenAI ##From the openai library, import the OpenAI class so we can use it in our program.

client = OpenAI() ##OpenAI = blueprint/class ; OpenAI() = create an object from that blueprint ; client = variable holding that object

response = client.responses.create (
   model = "gpt-5.6",
   input = "say hello to me in one sentence"
)

print(response.output_text)

## client → your OpenAI client object.
##responses → gives access to the Responses API .
##create(...) → makes the API request
##returned response → stored in the variable response
```

But at this point, you will need some OpenAI API credits to run any program.
Even `gpt-5.6` (which currently routes to GPT-5.6 Sol) costs $5 per million input tokens and $30 per million output tokens.

So, while its costs money, you can customize it to reduce cost. Use the cheapest model that still teaches the concept correctly.
For eg:
GPT-5.6 Sol: $5 / million input tokens, $30 / million output tokens.
GPT-5.6 Terra: $2 / million input, $12 / million output.
GPT-5.6 Luna: $0.20 / million input, $1.20 / million output.

I'd budget roughly **$20–50 USD total for the 100day program. You don't need hundreds of dollars to complete this curriculum**

Ok, once I got credits, I went ahead and ran the program. And it responds with a “Hello!”, since that’s what I asked the LLM to process and do.

My first Python → OpenAI API → LLM model → Python round trip is officially working.

Section 3 - Add a customer case scenario

First step is to create a prompt through the Python program. This will be later helpful in automating the Python to LLM and back pipeline.

Keep these:

```
from openai import OpenAI

client = OpenAI()
```

Now, **before the API call**, create a variable called:

```
customer_note = """
Customer: Acme Corp

The sales team spends around 8 hours every week manually
reading support tickets and copying important customer
problems into Salesforce. The VP of Sales thinks they're
missing important information and wants to automate it.
"""
```

Now we need to tell the model **what to do with `customer_note`**.

Replace your existing `"Say hello..."` API call with:

```
response = client.responses.create(
    model="gpt-5.6",
    input=f"""
Analyze the following customer note.

Identify:
1. Customer problem
2. Current process
3. Business impact
4. Potential AI solution

Customer note:
{customer_note}
"""
```

)

Keep:

print(response.output_text)

at the bottom.

When you run this program, it will return a response that will analyze the Customer note and present the analysis in the format that was requested.

Something like this -
1. **Customer problem** Important customer issues in support tickets may not be consistently identified or recorded in Salesforce.
2. **Current process** The sales team manually reads support tickets, identifies relevant customer problems, and copies them into Salesforce. This takes approximately 8 hours per week.
3. **Business impact** - Significant weekly time spent on manual data entry - Risk of missing critical customer information - Incomplete or inconsistent Salesforce records - Reduced sales productivity and visibility into customer needs
4. **Potential AI solution** Implement an AI-powered workflow that: - Automatically analyzes and summarizes support tickets - Identifies important customer problems, themes, urgency, and account details - Creates or updates relevant Salesforce records - Flags high-priority issues for sales review - Keeps a human approval step for uncertain or sensitive cases

Now, you've now built the complete loop:
customer_note
↓
Python constructs prompt
↓
client.responses.create()
↓
OpenAI API
↓
GPT-5.6
↓
response
↓
response.output_text
↓
Terminal

So basically, the prompt sent by the program goes through the API interface and is submitted to the LLM model as input. This input is then processed and the output of the LLM is passed back to the program through the API and then stored in the program.

So for the bigger picture you could : take a real-world business problem and write Python that:

Takes Business data

↓

Constructs a prompt

↓

Calls an AI model through an API

↓

Receives generated output

↓

Uses that output in your program